

SLAM

INF8422 : Perception Robotique et Intelligence Spatiale

Prof. Pierre-Yves Lajoie

**POLYTECHNIQUE
MONTREAL**



Agenda

1. Introduction au SLAM
2. Les Caméras
3. Odométrie Visuelle
4. IMU et Navigation Inertielle
5. SLAM Visuel-Inertiel (VIO)
6. Histoire du SLAM

Introduction au SLAM

Qu'est-ce que le SLAM ?

Simultaneous Localization And Mapping

- **Simultanée** : Les deux processus s'effectuent en même temps.
- **Localisation** : Estimer la pose du robot (position et orientation) dans l'environnement.
- **Cartographie** : Construire un modèle (une carte) de l'environnement.

Objectif fondamental

Rendre un robot **autonome** dans un environnement **inconnu**, sans avoir besoin d'infrastructure externe (GPS, balises, etc.).

La carte et la trajectoire sont **estimées conjointement** à partir des données des capteurs :

$$P(X, M \mid Z)$$

- $X = \{x_0, x_1, \dots, x_k\}$: Trajectoire (poses).
- $M = \{l_1, \dots, l_m\}$: Carte (landmarks).
- Z : Observations capteurs.

Le problème

Si je connais ma position exacte → construire la carte est facile (*mapping avec poses connues*).

Si je connais la carte exacte → me localiser est facile (*localisation sur carte connue*).

Le dilemme du SLAM

Nous ne connaissons **ni l'un ni l'autre** au départ.

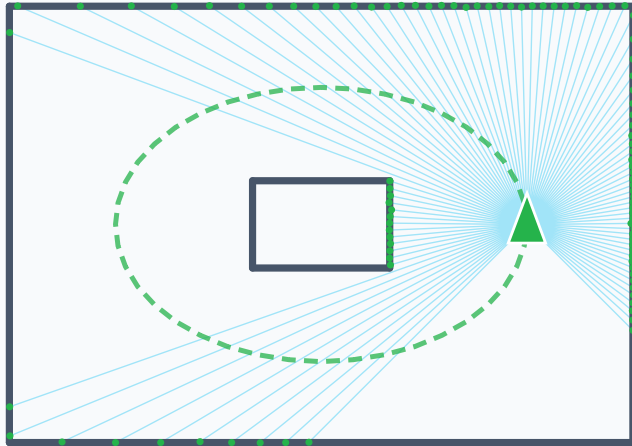
Les erreurs de localisation *corrompent* la carte, et les erreurs de carte *dégradent* la localisation.

Pourquoi est-ce difficile ?

1. **Bruit capteurs** : Aucune mesure n'est parfaite.
2. **Dérive** : L'incertitude s'accumule au fil du temps.
3. **Association de données** : Ce que je vois est-il un lieu connu ou nouveau ?
4. **Environnements dynamiques** : Le monde bouge.
5. **Complexité** : Le problème grossit avec le temps.

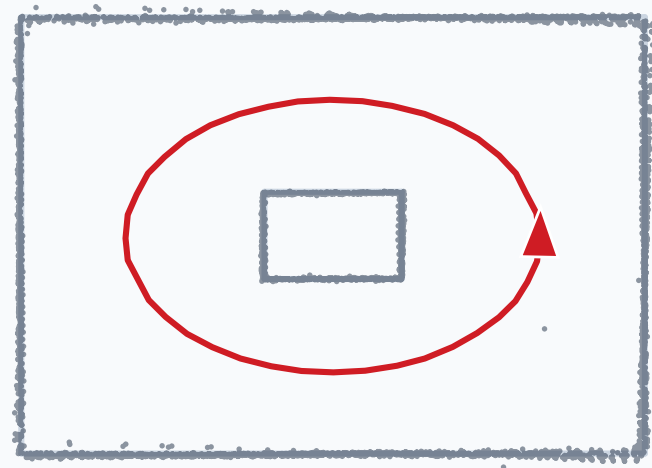
2D LiDAR SLAM

vérité terrain — le robot et son scan courant



--- trajectoire vraie faisceau lidar murs réels

carte estimée — scans posés aux poses estimées



— trajectoire estimée carte accumulée murs réels

$\sigma_{\text{scan}} = 0.01 \text{ m}$  FOV = 270° 96 rayons  $\sigma_{\text{odom}} = 0.03 \text{ m/pas}$     

k = 40 / 40 erreur de pose 0.05 m · épaisseur des murs 0.02 m

Entrées et Sorties du SLAM

Entrées – Données Capteurs

Proprioception (*mesures internes*) :

- Encodeurs de roues
- Unité de Mesure Inertielle (IMU)

Extéroception (*mesures de l'environnement*) :

- Caméras (mono, stéréo, RGB-D)
- LiDAR (3D, 2D)
- Radar, Sonar

Sorties – Estimations

Trajectoire du robot :

$$X = \{x_0, x_1, \dots, x_k\} \in SE(3)^k$$

Carte de l'environnement :

$$M = \{l_1, l_2, \dots, l_m\}$$

Points (sparse), voxels (dense), objets (sémantique)...

Navigation à l'aveugle vs SLAM

Dead Reckoning (sans carte)

- Intégration des mesures de mouvement (IMU, roues).
- Les erreurs s'accumulent avec le temps.

Sans correction extérieure, le robot finit par ne plus savoir où il est.

SLAM (avec fermetures de boucle)

- Utilise des repères de l'environnement pour corriger l'estimation.
- Chaque fois qu'un **lieu connu est reconnu**, l'erreur accumulée est **bornée**.
- La trajectoire et la carte sont **corrigées globalement**.

Exemple : l'aspirateur robot

En parcourant une pièce, il reconnaît qu'il repasse près du chargeur et recalcule sa position.

Fermeture de Boucle

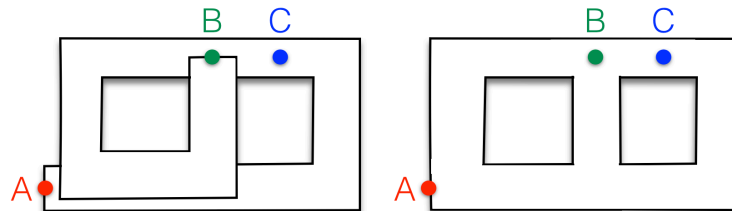
Une **fermeture de boucle** se produit quand le robot reconnaît un lieu **déjà visité**.

C'est le moment crucial où l'on peut :

- **Borner** l'erreur accumulée.
- **Corriger** toute la trajectoire passée.
- **Fusionner** deux parties de la carte.

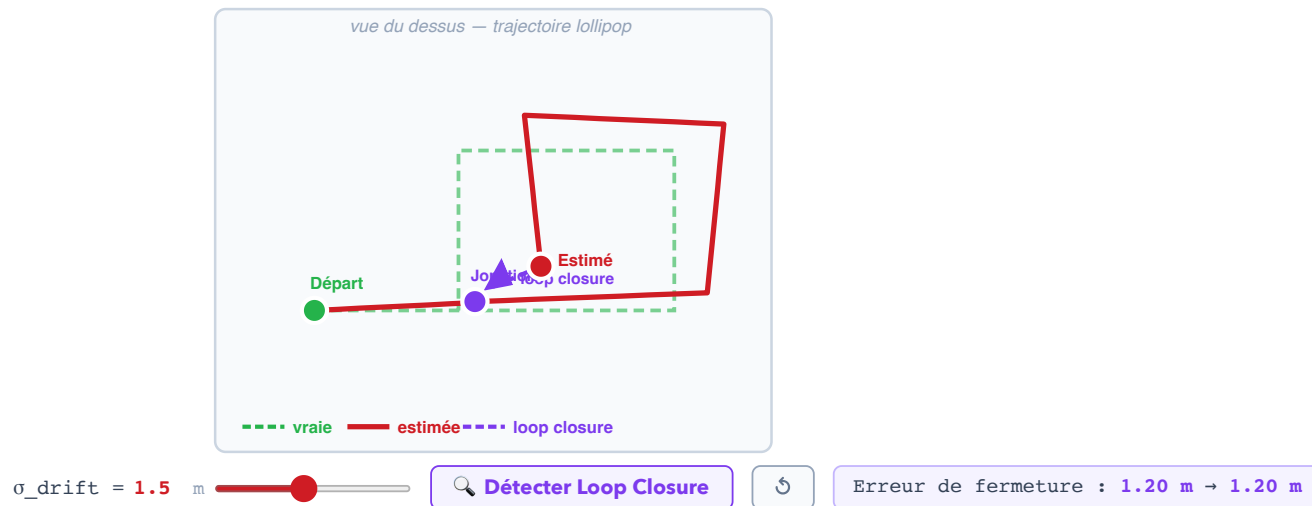
Sans fermeture de boucle

Le monde ressemble à un "couloir infini" : le robot ne réalise jamais qu'il est revenu à un point connu.



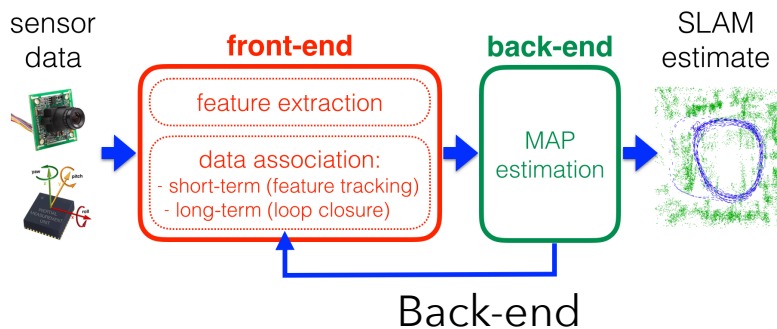
Sans vs avec fermeture de boucle

Fermeture de Boucle – Visualisation Interactive



Architecture : Front-end / Back-end

Le découpage standard de tout système SLAM moderne.



Front-end

Transforme les données brutes des capteurs en **contraintes géométriques** :

- Extraction de caractéristiques.
- Association de données (tracking, loop detection).
- Estimation de mouvement local.

Back-end

Reçoit les contraintes du Front-end et résout le **problème global** :

- Maximum A Posteriori (MAP).
- Optimisation de graphe de facteurs.
- Correction globale de trajectoire et carte.

Modalités de Capteurs pour le SLAM

Capteur	Points forts	Limites
Caméra mono	Légère, bon marché, riche en texture	Pas d'échelle absolue, sensible à la luminosité
Stéréo / RGB-D	Profondeur directe, échelle métrique	Portée limitée (RGB-D < 5m)
LiDAR 3D	Très précis, fonctionne dans l'obscurité	Coût élevé, sparse
IMU	6-DOF haute fréquence (100-1000 Hz)	Dérive, biais

Modalités de Capteurs pour le SLAM

Fusion	Avantage principal
Visual SLAM	Richesse texturale, compact
LiDAR SLAM	Précision géométrique absolue
VIO (Visual-Inertial)	Robustesse aux flous, fast motion
LIO (LiDAR-Inertial)	Précision + distorsion corrigée
Radar SLAM	Brouillard, pluie, nuit

Robustesse

Les systèmes les plus robustes combinent **plusieurs modalités** : VIO + LiDAR pour la conduite autonome, caméra + radar pour conditions extrêmes.

Représentations de la Carte

Clairsemée (Sparse)

Points d'intérêt / landmarks – suffisante pour la localisation, légère.

Exemple : ORB-SLAM3

Dense

Reconstruction 3D complète (mesh, voxels, TSDF) – nécessaire pour la navigation, manipulation, visualisation.

Exemples : KinectFusion, ElasticFusion

Sémantique

Objets étiquetés (chaise, porte, mur) – compréhension de la scène, landmarks stables pour la localisation à long terme.

Exemples : SemanticFusion, Kimera

Topologique

Graphe de lieux (nœud A connecté à nœud B) – navigation haut niveau, compact.

Le choix de la représentation dépend de la **tâche** : localisation → sparse ; planification → dense ; interaction → sémantique.

Ce cours : le Front-end visuel et inertiel

Section	Contenu	Rôle dans le système
2 – Caméras	Modèle pinhole, calibration, stéréo, RGB-D	Entrée sensorielle du Front-end
3 – Odométrie Visuelle	Features, géométrie, BA local	Estimation locale du mouvement
4 – IMU	Accéléromètre, gyroscope, intégration	Capteur proprioceptif haute fréquence
5 – VIO	Facteur IMU, fusion, observabilité	Pont Front-end → Back-end

Les Caméras

Modèle Pinhole : La projection perspective

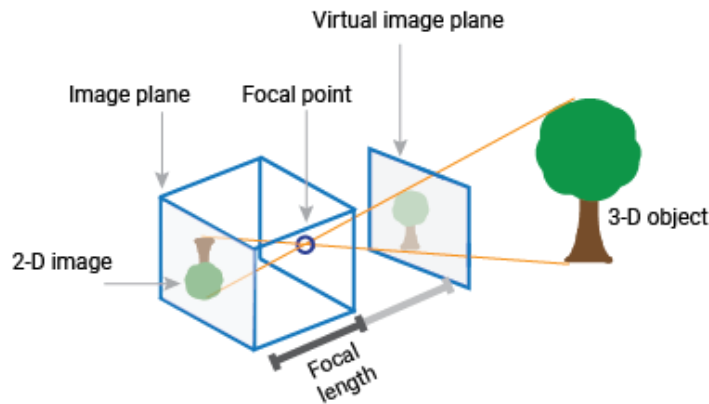
Le modèle sténopé (pinhole) décrit comment un point 3D se projette sur le plan image.

Principe : L'image se forme à une distance focale F (en mm) du point focal.

Projection : Soit $p^c = [p_x^c, p_y^c, p_z^c]^T$ un point exprimé dans le repère caméra (c). Ses coordonnées physiques sur le capteur sont :

$$x = F \frac{p_x^c}{p_z^c}, \quad y = F \frac{p_y^c}{p_z^c}$$

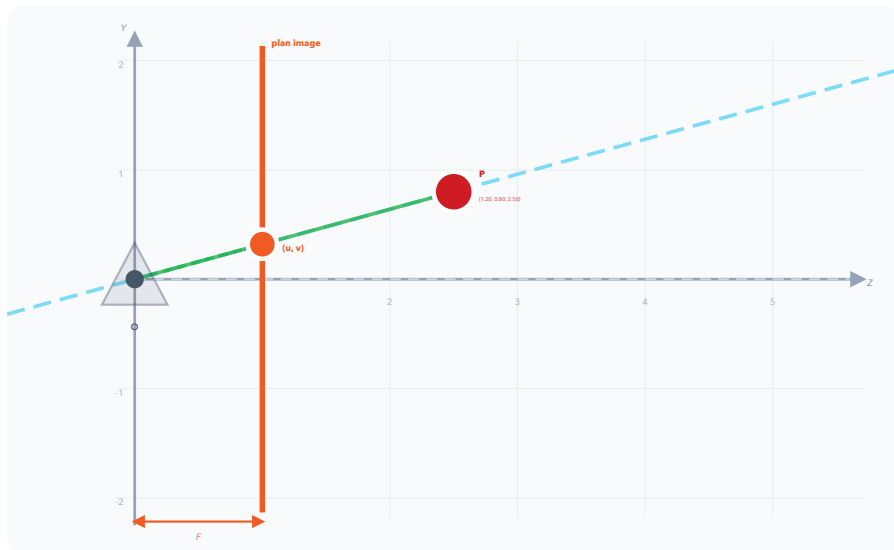
Note : x et y sont des distances physiques (mm), pas encore des pixels.



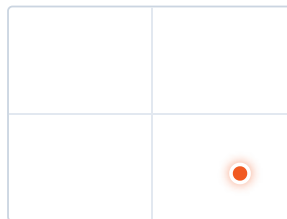
Projection géométrique (Source: MathWorks)

C'est la **projection perspective** : les objets lointains apparaissent petits. L'information de profondeur est **perdue** dans la projection.

Modèle Pinhole – Projection Interactive



Plan image (640 × 480 px)



(512, 368) px

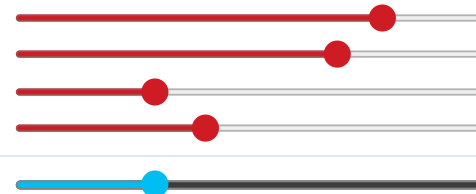
X = 1.20

Y = 0.80

Z = 2.50

f = 400 px

$\lambda = 1.00$



← déplacer P le long du rayon

$$u = f \cdot \frac{X}{Z} + c_x = 400 \cdot \frac{1.20}{2.50} + 320 = 512$$

$$v = f \cdot \frac{Y}{Z} + c_y = 400 \cdot \frac{0.80}{2.50} + 240 = 368$$

Δ λ s'annule : (u,v) ne change pas avec λ .
La profondeur Z est perdue.

Paramètres Intrinsèques : Caméra $\rightarrow$ Pixels (u, v)

Conversion des coordonnées physiques (mm) vers les pixels (u, v) .

On convertit la focale F (mm) en pixels via la taille des pixels (p_u, p_v) :

$$f_u = \frac{F}{p_u}, \quad f_v = \frac{F}{p_v}$$

Matrice Intrinsèque K :

$$\underbrace{\begin{bmatrix} f_u & 0 & c_u \\ 0 & f_v & c_v \\ 0 & 0 & 1 \end{bmatrix}}_K \begin{bmatrix} p_x^c/p_z^c \\ p_y^c/p_z^c \\ 1 \end{bmatrix}$$

- (f_u, f_v) : Focales en pixels.
- (c_u, c_v) : Point principal, centre optique de l'image en pixels.

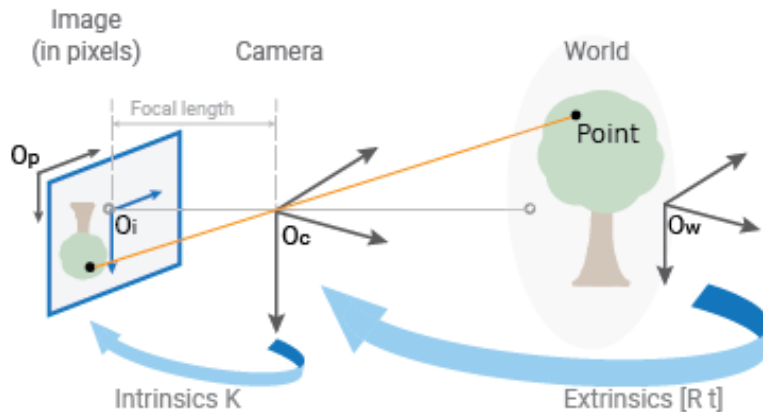
Paramètres Extrinsèques : Monde → Caméra

Passage du repère monde (w) au repère caméra (c).

Transformation rigide composée d'une rotation R_w^c et d'une translation t_w^c :

$$\underbrace{\begin{bmatrix} R_w^c & t_w^c \\ 0 & 1 \end{bmatrix}}_{T_w^c} \begin{bmatrix} p^w \\ 1 \end{bmatrix}$$

- R_w^c (3×3), t_w^c (3×1) : rotation et translation **monde** → **caméra**.
- La **pose de la caméra dans le monde** est l'inverse (cours 1) : $T_c^w = (T_w^c)^{-1}$, soit $R_c^w = (R_w^c)^T$ et $t_c^w = -(R_w^c)^T t_w^c$.
- En **SLAM** : c'est cette pose T_c^w (ou son inverse T_w^c) qui est **l'inconnue à estimer**.



Repère Monde vs Repère Caméra

Fonction d'observation $h(\cdot)$

Dans le SLAM visuel, la **fonction d'observation** relie un point 3D du monde à un pixel dans l'image :

Étape 1 de $h(\cdot)$ – Extrinsèques (Monde $\rightarrow$ Caméra) :

$$p^c = R_w^c p^w + t_w^c$$

On peut donc modéliser une observation :

$$\bar{z} = h(T_c^w, p^w) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \Sigma)$$

Étape 2 de $h(\cdot)$ – Intrinsèques (Caméra $\rightarrow$ Pixels) :

$$\begin{bmatrix} u \\ v \end{bmatrix} = \underbrace{\begin{bmatrix} f_u & 0 & c_u \\ 0 & f_v & c_v \\ 0 & 0 & 1 \end{bmatrix}}_K \begin{bmatrix} p_x^c/p_z^c \\ p_y^c/p_z^c \\ 1 \end{bmatrix} = \begin{bmatrix} f_u \frac{p_x^c}{p_z^c} + c_u \\ f_v \frac{p_y^c}{p_z^c} + c_v \end{bmatrix}$$

- T_c^w : Pose caméra (paramètre extrinsèque).
- p : Position 3D du landmark.
- $\bar{z} = (u, v)$: Observation en pixels.
- ϵ : bruit.

Reprojection Error

L'erreur de reprojection pour un point est $\|\bar{z} - h(T_c^w, p^w)\|^2$. Le Back-End (prochain cours) va ensuite trouver les $T_{c_i}^w$ et p_i^w qui minimise la somme des erreurs de reprojection pour tous les points :

Réalité physique : Les Distorsions

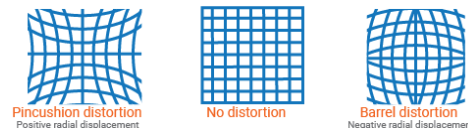
Les lentilles réelles ont des imperfections optiques, **il faut corriger avant toute estimation.**

1. Distorsion Radiale

- Causée par la courbure de la lentille.
- Les rayons sont déviés davantage **loin du centre.**
- **Conséquence** : Les lignes droites apparaissent **courbes.**
- **Effets** : "Barillet" (GoPro, fisheye) ou "Pincushion".

Ces défauts sont constants → corrigés par calibration. On trouve les paramètres de calibration en observant un échiquier de taille connu. Ensuite on corrige l'image avec par exemple OpenCV

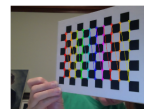
```
cv2.undistort .
```



Distorsion radiale en barillet (à gauche) et pincushion (à droite)

2. Distorsion Tangentielle

- Causée par un défaut d'assemblage.
- La lentille n'est pas parfaitement parallèle au capteur.



Calibration avec un patron connu. On compare les distances observées sur l'image avec les distances réelles entre les coins de l'échiquier.

Vision Stéréo – Disparité

Comment retrouver la profondeur Z à partir de **deux images** ?

Configuration standard :

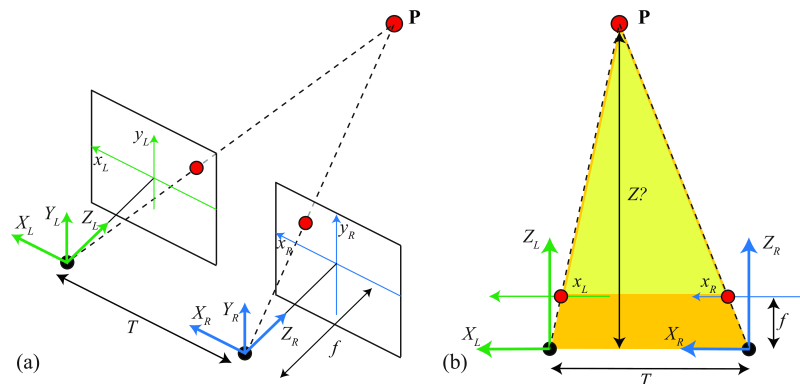
- Deux caméras identiques, axes optiques **parallèles**.
- Séparées par une distance b (**baseline**).
- Focale f (en pixels).

Disparité d : Différence de position horizontale d'un même point P dans les deux images :

$$d = u_L - u_R$$

Un objet **proche** se déplace beaucoup entre les deux images → grande disparité.

Un objet **lointain** se déplace peu → petite disparité.



Géométrie stéréo standard (Source: MIT Vision Book)

Vision Stéréo – Profondeur

Relation de profondeur Z : Par triangles semblables
(Thalès) :

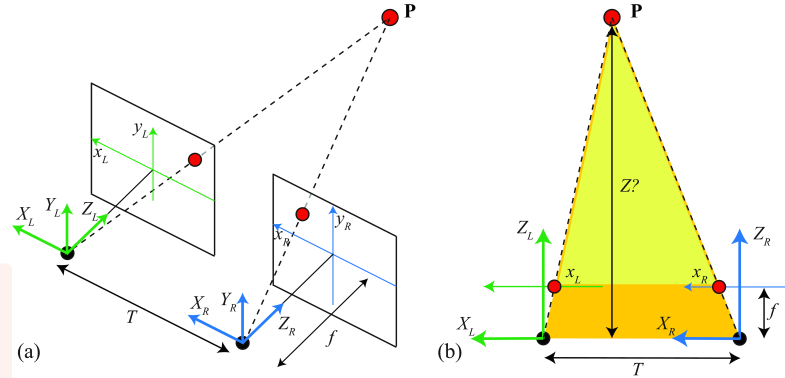
$$\frac{b}{Z} = \frac{d}{f} \implies \boxed{Z = \frac{f \cdot b}{d}}$$

Intuition clé

La profondeur est **inversement proportionnelle** à la disparité.

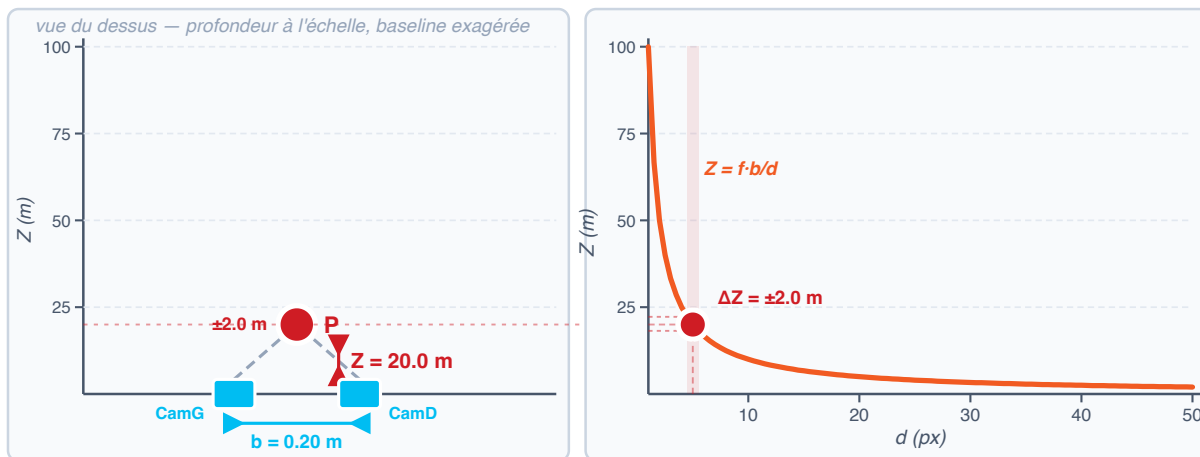
- Grande $d \rightarrow$ objet proche.
- Petite $d \rightarrow$ objet lointain (et plus incertaine).

Pour voir loin avec précision, il faut une **grande baseline** b (appelée T dans la figure.).



Géométrie stéréo standard (Source: MIT Vision Book)

Vision Stéréo – Visualisation Interactive



$f = 500$ px $b = 0.20$ m $d = 5$ px

incertitude sur $d = 0.5$ px

$z = f \cdot b / d = 20.0$ m $\Delta z = \pm 2.02$ m (10 %)

pente $|dz/dd| = z^2 / (f \cdot b) = 4.00$ m/px

Capteurs RGB-D – Profondeur Active

Alternatives à la stéréoscopie passive pour mesurer Z directement.

1. Stéréo IR Actif (ex : Intel RealSense D435)

① **Projecteur IR** – projette un motif de points invisibles à l'œil nu sur la scène.

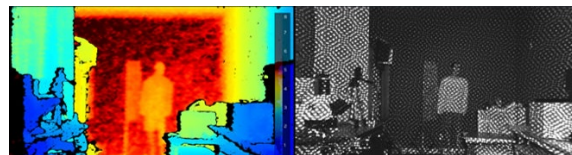
② **Deux caméras IR** – voient le motif simultanément ; comme deux yeux, elles perçoivent chaque point légèrement décalé.

③ **Calcul de profondeur** – le processeur mesure le *décalage* de chaque point entre les deux images $\rightarrow Z = f \cdot b/d$.

- **Limite** : IR saturé en plein soleil.

3. Time-of-Flight (ToF) (ex : Kinect v2)

- Mesure le déphasage d'une lumière modulée $\rightarrow$ profondeur directe.



Intel RealSense D435 – stéréo IR active (Source: Intel)

Avantage pour le SLAM

- Profondeur directe $\rightarrow$ pas d'ambiguïté d'échelle en monoculaire.
- Facilite l'association de points, ne nécessite pas de features.
- Très utilisé en intérieur (robot domestique, scan de bâtiment).

Odométrie Visuelle

Pipeline Feature-Based – Vue d'ensemble

La méthode classique, encore dominante en pratique (ORB-SLAM, VINS-Mono) :

① Détection

Trouver des *interest points* (coins, blobs) dans l'image I_k .

② Description

Calculer un vecteur descripteur robuste autour de chaque point.

③ Matching

Trouver les correspondances entre I_{k-1} et I_k .
Filtrer avec RANSAC.

④ Estimation

Calculer $T_{k-1,k}$ par contrainte de coplanarité (2D-2D) ou PnP (3D-2D).

Similaire à ICP

On associe les pixels des deux images (étapes 1 à 3).

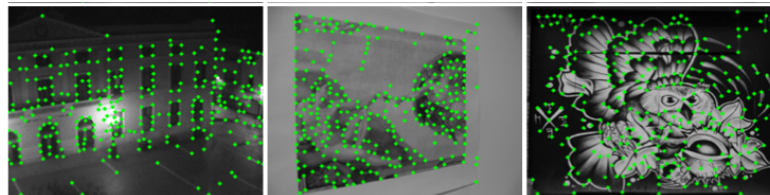
Puis, on calcule la rotation et translation entre les deux images.

Feature Detectors – Détecter les Points d'Intérêt

On veut des points **répétables** et **stables** sous rotation, changement d'échelle et d'illumination.

FAST (*Features from Accelerated Segment Test*)

- Test binaire sur un cercle de 16 pixels.
- Détecte des coins dans les images.
- **Très rapide** pour le temps réel.



Keypoints détectés sur une image

Autres détecteurs :

- **Harris** : réponse au coin, plus lent.
- **SIFT / SURF** : invariant à l'échelle, coûteux.
- **ORB** = FAST + orientation → **rapide et invariant**.

Trouver les Correspondances

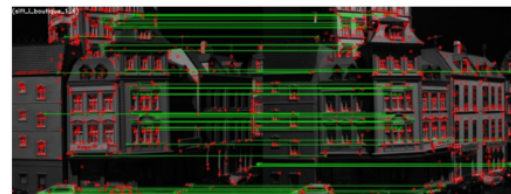
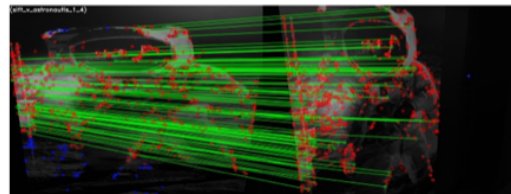
Comment savoir que le point A dans I_1 est le point B dans I_2 ?

BRIEF / ORB Descriptor :

- Vecteur **binaire** (128–256 bits) construit par comparaisons d'intensité par paires autour du keypoint.
- Matching via **distance de Hamming** (XOR bit-à-bit) → très rapide.
- **Inconvénient majeur**: les descripteurs étant imparfaits, ils produisent beaucoup de mauvaises correspondances.

Filtrage des mauvaises correspondances – RANSAC :

1. Choisir **N points** au hasard.
2. Calculer le modèle. 3. Compter les **inliers**.
4. Répéter → garder le **meilleur modèle**.



Matchings bruts (haut) et après RANSAC (bas)

Matching & RANSAC – Visualisation Interactive

Image k-1

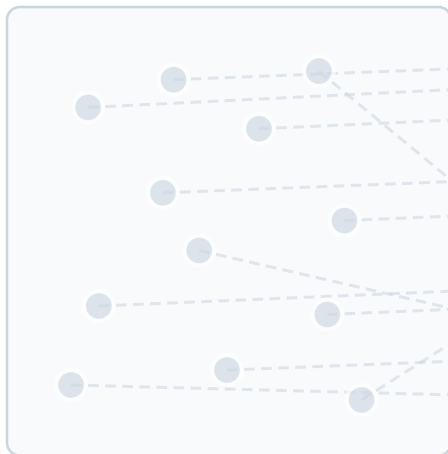


Image k



① Tirer 2 points



② Calculer modèle



③ Mesurer l'accord



④ Garder le meilleur

12 correspondances brutes : impossible de savoir lesquelles sont fausses



1 / 11

Trouver la transformation entre deux images : Contrainte de Coplanarité

Utilisée pour l'**initialisation** (pas encore de carte 3D) – entrée : correspondances 2D-2D .

Le même point 3D, vu deux fois (*coordonnées normalisées*)
 $x = K^{-1}\tilde{m}$:

$$\lambda_1 x_1 = t_2^1 + \lambda_2 R_2^1 x_2$$

Les profondeurs λ_1, λ_2 sont **inconnues**, mais x_1, t_2^1 et $R_2^1 x_2$ sont dans un **même plan** : leur produit mixte est nul.

$$x_1^T (t_2^1 \times R_2^1 x_2) = 0 \quad \Longleftrightarrow \quad \boxed{x_1^T E x_2 = 0}$$

Matrice essentielle $E = [t_2^1]_{\times} R_2^1$: chaque correspondance donne **une** équation. Plusieurs correspondances $\rightarrow E \rightarrow R$ et la **direction** de t .

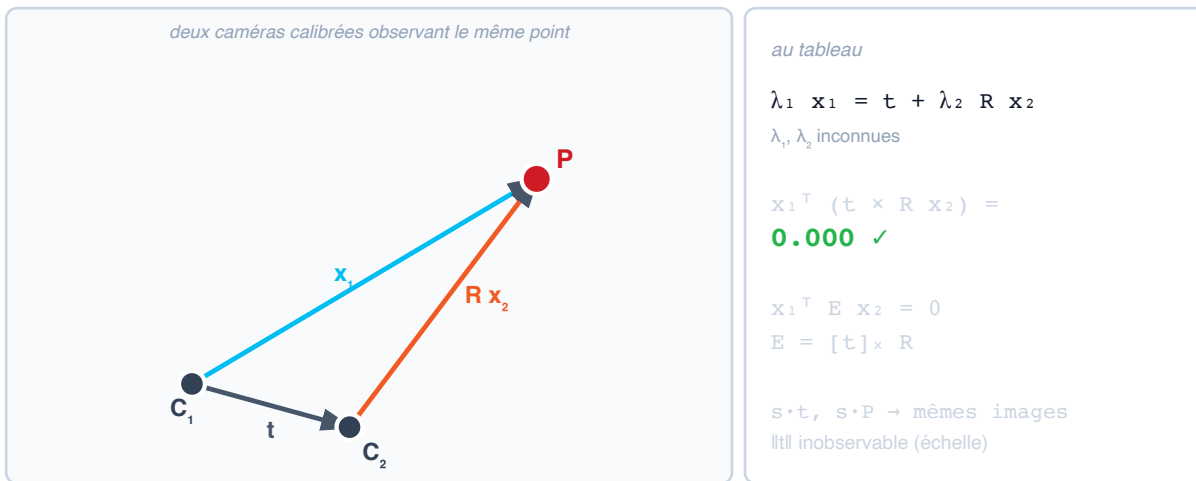
La procédure :

correspondances $(x_1, x_2) \rightarrow E \rightarrow R, t$

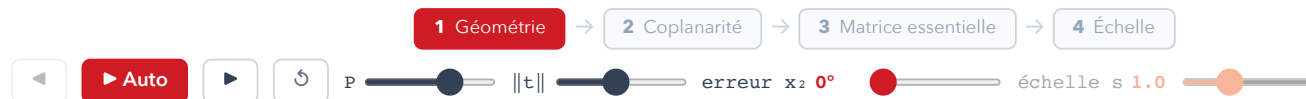
Chaque correspondance apporte **une** contrainte sur la pose relative. Avec plusieurs correspondances on estime E , puis on la **décompose** pour retrouver R et la **direction** de t .

```
# OpenCV – Matrice Essentielle + pose
E, mask = cv2.findEssentialMat(
    pts1, pts2, K, method=cv2.RANSAC)
_, R, t, _ = cv2.recoverPose(E, pts1, pts2, K)
# t est unitaire : pas d'échelle absolue
```


Contrainte de Coplanarité – Visualisation Interactive



Le même point 3D vu deux fois : $\lambda_1 \mathbf{x}_1 = \mathbf{t} + \lambda_2 \mathbf{R} \mathbf{x}_2$. Les deux profondeurs sont inconnues.



3D-2D : Perspective-n-Point (PnP)

Utilisé pour le **tracking** (carte 3D disponible) – entrée : points 3D de la carte + pixels 2D détectés.

D'où viennent les points 3D ? Triangulés lors de l'initialisation (2D-2D) et ajoutés à la carte.

Pour chaque correspondance, en coordonnées normalisées $x_i = K^{-1}\tilde{m}_i$:

$$\lambda_i x_i = R_w^c p_i^w + t_w^c$$

L'amer ramené dans le repère caméra doit tomber **sur le rayon** de son observation ; sa profondeur λ_i reste inconnue.

- En pratique : **solvePnP**Ransac → robuste aux faux positifs.

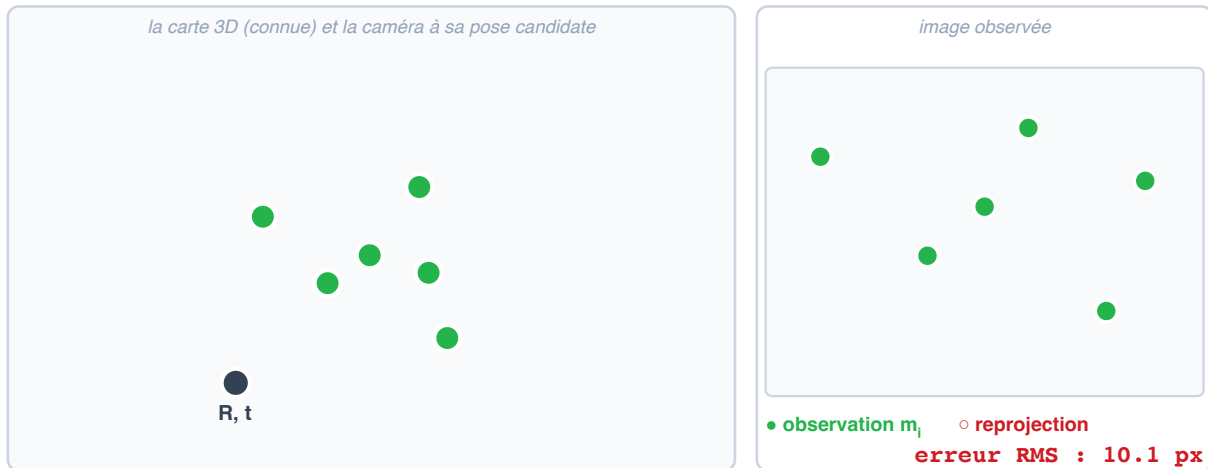
Si les points 3D sont en mètres (ex: via stéréo) → pose avec **échelle métrique**. Si les points 3D sont normalisés (monoculaire) → pose sans échelle (*up-to-scale*).

```
# OpenCV – PnP robuste avec RANSAC
ret, rvec, tvec, inliers = cv2.solvePnP(
    pts3d,          # (N,3) float32 – carte
    pts2d,          # (N,2) float32 – pixels
    K, dist_coeffs)

# Convertir vecteur rotation → matrice
R, _ = cv2.Rodrigues(rvec)
# T = [ R | t ]
```

	2D-2D	PnP (3D-2D)
Entrée	2D ↔ 2D	3D ↔ 2D
Équation	$x_1^T E x_2 = 0$	$\lambda_i x_i = R_w^c p_i^w + t_w^c$
Sortie	pose relative	pose dans la carte
Échelle	inconnue	celle de la carte
Usage	initialisation	tracking

PnP – Minimiser la différence entre la pose estimée de la caméra et les observations.



Connus : les amers 3D de la carte et leurs pixels. Inconnue : la pose (R_w^c , t_w^c) de la caméra.

1 Poser le problème → 2 Modèle de projection → 3 Pourquoi plusieurs points → 4 Erreur de reprojection

◀ ▶ t_x t_z lacet ▶ Minimiser l'erreur ↺ déréglé

PnP – Contrainte Géométrique

Légende de l'animation :

Symbole	Signification
p_i^w	amer 3D de la carte – connu
rayon bleu	direction d'observation $x_i = K^{-1}\tilde{m}_i$, issue du pixel
segment rouge (3D)	écart entre l'amer et son rayon : ce que la pose doit annuler
● vert	observation m_i dans l'image
O rouge	reprojection $\pi(K(R_w^c p_i^w + t_w^c))$
segment rouge (image)	résidu e_i

Objectif – minimiser l'erreur de reprojection :

$$\min_{R_w^c, t_w^c} \sum_{i=1}^N \|m_i - \pi(K(R_w^c p_i^w + t_w^c))\|^2$$

où $\pi(X, Y, Z) = (X/Z, Y/Z)$ est la projection perspective, sans la profondeur λ_i .

Chaque correspondance (p_i^w, m_i) donne **2 équations** (u, v). La pose a **6 DDL** → il faut au moins 3 paires.

```
ret, rvec, tvec, inliers = cv2.solvePnPRansac(  
    pts3d, pts2d, K, dist)  
R, _ = cv2.Rodrigues(rvec)
```

Méthodes Directes – Alternative aux Features

Au lieu d'extraire des descripteurs, on aligne directement l'**intensité des pixels** entre images.

- Pas d'extraction de keypoints → exploite toute la texture, même faible.
- Précision sub-pixel, mais sensible aux changements d'illumination.
- Besoin de **pyramides d'images** (coarse-to-fine) pour la convergence.
- Produisent des cartes **semi-denses** (tous les pixels à gradient fort).

Systèmes directs notables

- **DSO** (Direct Sparse Odometry) – sparse direct, état de l'art vitesse/précision
- **LSD-SLAM** (Large-Scale Direct) – semi-dense, loop closure
- **SVO** (Semi-Direct VO) – hybride : features pour tracking, photométrie pour profondeur

À retenir

Les méthodes directes ne remplacent pas les features dans tous les cas : elles excellent en intérieur texturé, mais peinent sous éclairage variable ou en extérieur.

Sélection de Keyframes

Pas besoin de traiter **chaque image** – on sélectionne des **keyframes** : les images les plus informatives.

Pourquoi des keyframes ?

- Images consécutives sont très similaires → redondance inutile.
- Le BA n'est lancé que sur les keyframes → coût contrôlé.

Critères de sélection :

1. **Parallaxe** : déplacement suffisant depuis la dernière keyframe ($> \theta_{min}$).
2. **Tracking** : nombre de features trackées tombe en dessous d'un seuil.
3. **Temps** : intervalle minimal entre keyframes respecté.

Cadence typique

À 30 fps, on sélectionne ~1 image sur 5-10 comme keyframe.

Compromis

Trop de keyframes → BA coûteux, lent. Trop peu → manque de contraintes, dérive.

VO : Composition de Poses et Dérive

L'odométrie visuelle estime le **mouvement relatif**

$\hat{T}_{k-1,k}$ entre deux frames consécutives, puis

compose les poses :

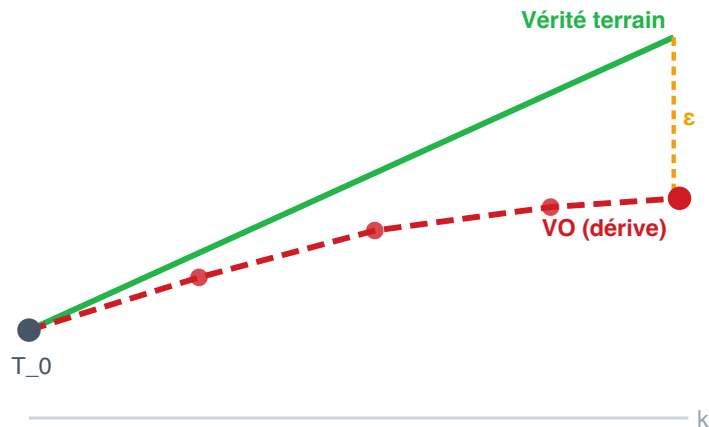
$$T_k = T_{k-1} \cdot \hat{T}_{k-1,k}$$

Chaque estimation $\hat{T}$ contient une petite erreur ϵ_k .

En composant, ces erreurs **s'accumulent**, la trajectoire dérive de la réalité.

Dérive du VO

L'erreur croît à chaque nouvelle image. Sans correction externe, le robot ne sait plus où il est après quelques centaines de mètres.



Dérive et Bundle Adjustment Local

Le problème : chaque estimation $\hat{T}_{k-1,k}$ a une petite erreur → compositions successives → **dérive** (*drift*).

Bundle Adjustment local : réoptimise simultanément les **K derniers keyframes** et leurs landmarks visibles :

$$\min_{T_j, P_i} \sum_{j \in \mathcal{W}} \sum_i \|z_{ij} - h(T_j, P_i)\|^2$$

- La fenêtre $\mathcal{W}$ glisse avec la caméra ($K \sim 10$).
- Coût $O(K^2)$ parce qu'on doit comparer chaque paire.

Coût vs précision

Plus la fenêtre est grande, plus la correction est précise, mais le coût augmente. En pratique $K \leq 10$ pour du temps réel.

Approche	Précision	Coût calcul
VO simple (PnP seul)	Faible (drift ↑)	Très bas
VO + BA local	Moyenne	Modéré
VO + BA global ($W = \infty$)	Haute	Élevé

Bundle Adjustment (BA)

Bundle Adjustment (BA) : Optimise simultanément les poses T_j et les points 3D P_i :

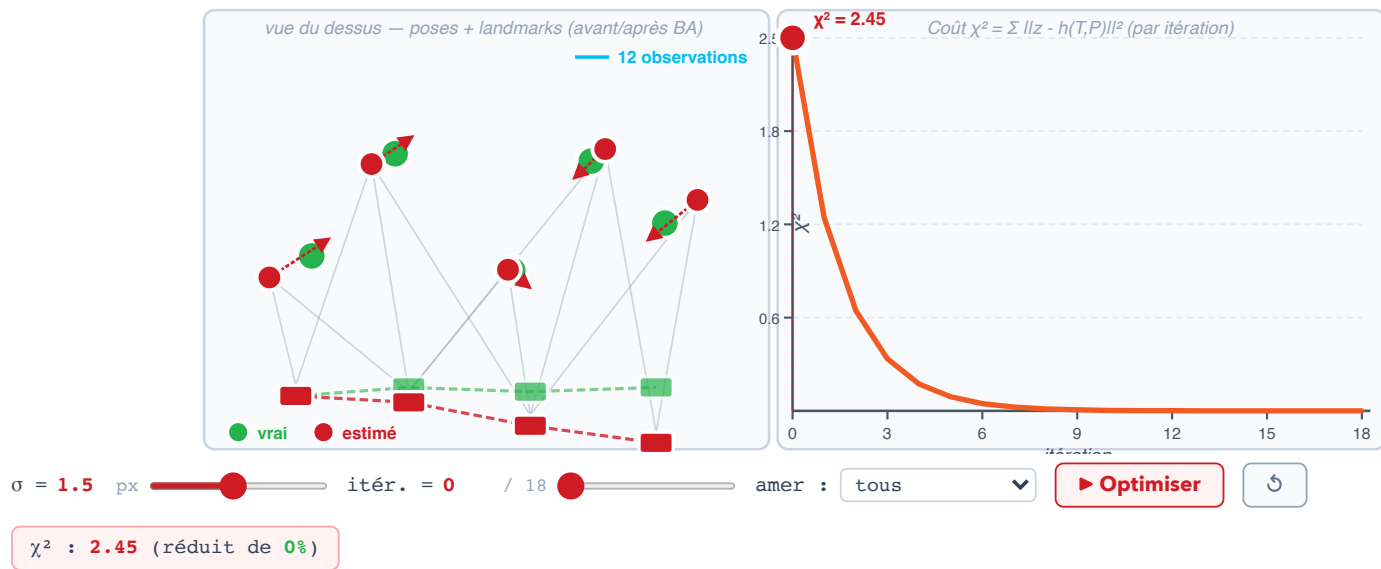
$$\min_{T,P} \sum_{i,j} \|z_{ij} - h(T_j, P_i)\|_{\Sigma_{ij}}^2$$

En pratique

Le BA **local** (fenêtre de keyframes) tourne en temps réel dans le Front-end.

Le pose graph optimization avec toute la trajectoire (BA **global**) tourne dans le Back-end après chaque loop closure, au prochain cours.

Bundle Adjustment – Visualisation Interactive



Limites du Visual SLAM – Pourquoi ajouter l'IMU ?

Limitations du V-SLAM seul

1. **Flou de mouvement** : à haute vitesse, les features deviennent inexploitable.
2. **Ambiguïté d'échelle** : en monoculaire.
3. **Occultation** : face à un mur blanc ou dans le noir, le système perd le tracking.
4. **Roll & Pitch** : une caméra seule ne peut pas mesurer l'orientation absolue par rapport à la gravité.

Solution : ajouter un capteur proprioceptif à haute fréquence.

L'**IMU** (Inertial Measurement Unit) mesure :

- Accélérations linéaires
- Vitesses angulaires

Il continue de fonctionner (> 200 Hz) **dans le noir, le brouillard, face à un mur blanc.**

Visual-Inertial Odometry (VIO)

La fusion caméra + IMU résout simultanément l'échelle, le roll/pitch et la robustesse aux mouvements rapides.

IMU et Navigation Inertielle

Unités de Mesure Inertielle (IMU) – Accéléromètre

Capteurs MEMS (Micro-Electro-Mechanical Systems). Dérivent rapidement.

Modèle de l'Accéléromètre (a_m)

Mesure la **force spécifique** dans le repère du robot (r) :

$$a_m^r = R_w^r (a^w - g^w) + b_a + n_a$$

- a^w, g^w : Accélération réelle et gravité (repère monde w).
- $R_w^r = (R_r^w)^T$: Rotation monde $\rightarrow$ robot.
- b_a, n_a : Biais (lentement variable) et Bruit blanc Gaussien.

Point clé

L'accéléromètre mesure la **force spécifique** $f = a - g$, **pas** l'accélération pure. Au repos sur une table, il mesure $+g$ vers le haut (réaction normale). En chute libre, il mesure **zéro**.

Unités de Mesure Inertielle (IMU) – Gyroscope

Modèle du Gyroscope (ω_m)

Mesure la vitesse angulaire du robot dans son propre repère :

$$\omega_m^r = \omega^r + b_g + n_g$$

- ω^r : Vitesse angulaire réelle (exprimée dans le repère du robot r).
- b_g, n_g : Biais du gyroscope et Bruit blanc Gaussien.

Utilisé pour intégrer la rotation :

$$R_{k+1} = R_k \cdot \text{Exp}((\omega_k - b_g) \Delta t)$$

Le gyroscope donne une **orientation relative** très précise sur de courtes durées.

Nous verrons la carte Exp (opération matricielle) au prochain cours.

Complémentarité Caméra / Gyroscope

La caméra donne une orientation **absolue** (par rapport à la scène) mais à basse fréquence.

Le gyroscope donne une orientation **relative** à très haute fréquence.

IMU : Bruit et Biais – Pourquoi ça dérive ?

1. Bruit Blanc (n) :

$$n \sim \mathcal{N}(0, \sigma^2)$$

Lors de l'intégration, le bruit cause un *Random Walk* en position :

$$\sigma_p(t) \propto \sigma_n \sqrt{t}$$

2. Biais (b) : Erreur systématique qui évolue lentement (Random Walk du biais) :

$$\dot{b}(t) = \eta_b(t), \quad \eta_b \sim \mathcal{N}(0, \sigma_b^2)$$

Dérive quadratique en position

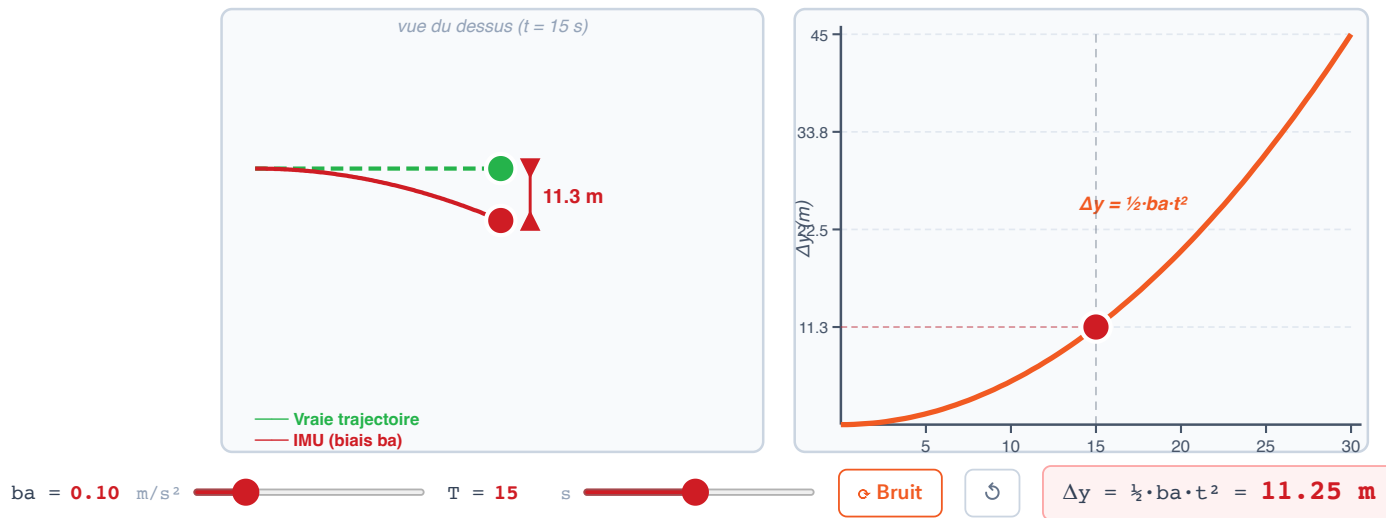
Un biais constant de seulement **0.1 m/s²** sur l'accéléromètre cause une erreur de position :

$$x(t) = \frac{1}{2} b \cdot t^2$$

Après **10 secondes** → erreur de **5 mètres** !

L'estimation des biais b_a, b_g est **critique** et est réalisée conjointement avec la pose dans le Back-end.

Dérive IMU – Visualisation Interactive



Cinématique Discrète

Si on connaît l'état $x_k = (R_k, v_k, p_k)$ et les mesures (ω_k, a_k) , on prédit x_{k+1} :

Rotation :

$$R_{k+1} = R_k \cdot \text{Exp}((\omega_k - b_g) \Delta t)$$

Vitesse linéaire :

$$v_{k+1} = v_k + (R_k (a_k - b_a) + g) \Delta t$$

Position :

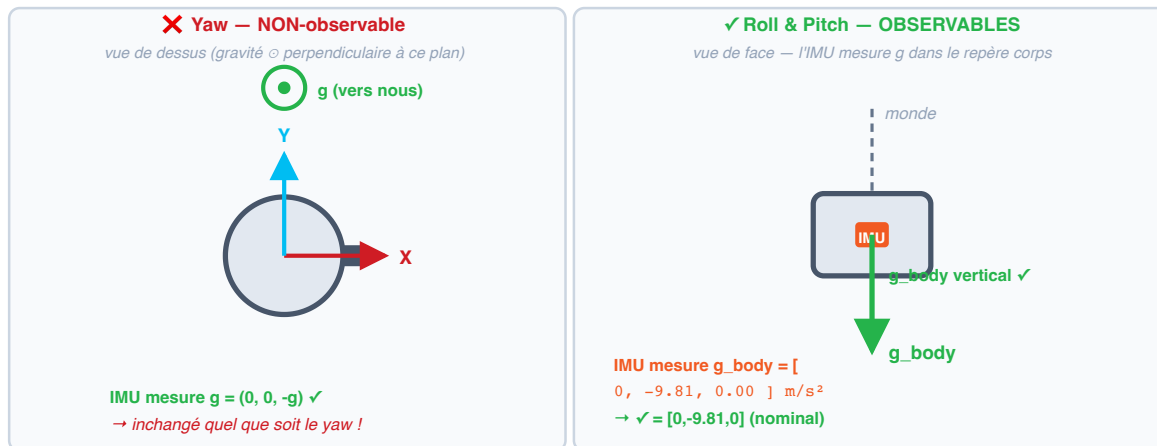
$$p_{k+1} = p_k + v_k \Delta t + \frac{1}{2} (R_k (a_k - b_a) + g) \Delta t^2$$

À retenir

- L'IMU fonctionne à **très haute fréquence** (200-1000 Hz).
- La caméra fonctionne à basse fréquence (10-30 Hz).
- Entre deux images, on intègre **des dizaines à des centaines** de mesures IMU.

SLAM Visuel-Inertiel (VIO)

Observabilité IMU



Yaw = 0°

non-obs.

Roll = 0°

observable

✗ non-observable : Tx, Ty, Tz, Yaw (4 DOF)

✓ observable : Roll, Pitch, Échelle (IMU)

Avantages du VIO

Roll & Pitch Observables

La gravité g est une référence absolue mesurée par l'accéléromètre.

→ Le roll et le pitch sont **observables**.

Échelle Métrique

En monoculaire, la caméra estime t en **unités relatives**.

L'accéléromètre mesure en **m/s²** → il fixe l'échelle métrique réelle.

→ L'ambiguïté d'échelle du monoculaire est **résolue**.

Robustesse

L'IMU fonctionne :

- Dans le **noir**.
- Face à un **mur blanc**.
- Lors de **mouvements rapides** (flou de mouvement).

Systèmes VIO populaires

ORB-SLAM3 (tightly coupled, multi-map) · **VINS-Mono** (préintégration) · **Kimera** (sémantique + VIO)

Graphe de Facteurs – Intuition

Structure qui **organise l'optimisation** dans le Back-end.

Un **graphe de facteurs** représente le problème d'estimation :

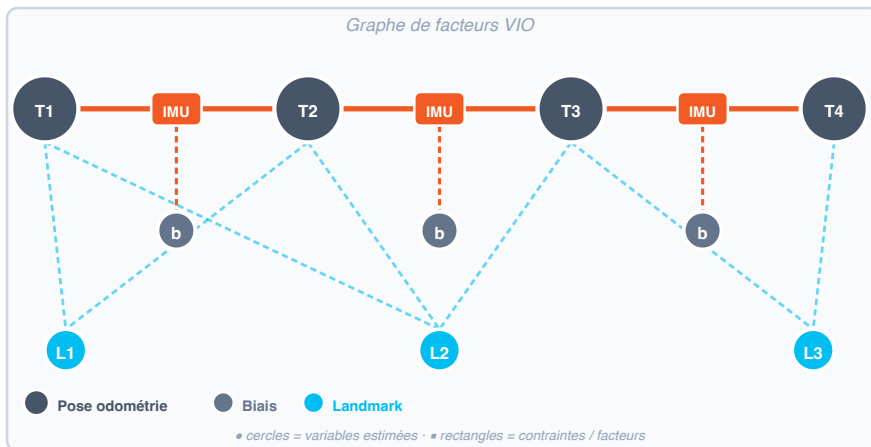
- **Nœuds (variables)** : ce qu'on veut estimer poses T_i , vitesses v_i , biais IMU b_i
- **Facteurs** : les contraintes imposées par les mesures: odométrie, reprojection, IMU
- **Optimisation** : trouver les valeurs des variables qui minimisent l'erreur de toutes les contraintes simultanément

Chaque facteur ajoute une **contrainte** (erreur à minimiser). Le **Back-end** (GTSAM, g2o) résout l'ensemble simultanément.

Cours suivant

Comment résoudre ce système efficacement ?
Moindres carrés, sparsité, Lie groups.

Graphe de Facteurs VIO – Visualisation Interactive



N keyframes = 4



